

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 02 — PLC PROGRAMMING LANGUAGES

Student Edition

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 02: PLC Programming Languages

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 2–3 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Compare the PLC programming languages presented in the course.
- Interpret a motor start/stop latching circuit in Ladder Diagram.
- Recognize equivalent motor-control logic represented using Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- Explain how tags and Boolean data types support PLC logic.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Start from the clean ICDFA PLC VM snapshot and sign in to the PLC training environment.

Step 2. Launch OpenPLC Editor and open the Ladder Diagram example from the PLC Programming Language Examples directory.

Step 3. Open Res0 > instance0 (Motor_Control). Identify the normally open Start contact, normally closed Stop contact, MotorOutput latch contact and MotorOutput coil.

Step 4. Open the tag/variable list. Confirm that the motor-control tags use Boolean values and record the purpose of each tag in your worksheet.

Step 5. Return to File > Open and load the Function Block Diagram example. Open Motor_Control_FBD and identify the block that maintains the motor output until a stop condition occurs.

Step 6. Open the Structured Text example and inspect MotorControlST. Describe how the same start/stop behavior is represented using text rather than graphical contacts and coils.

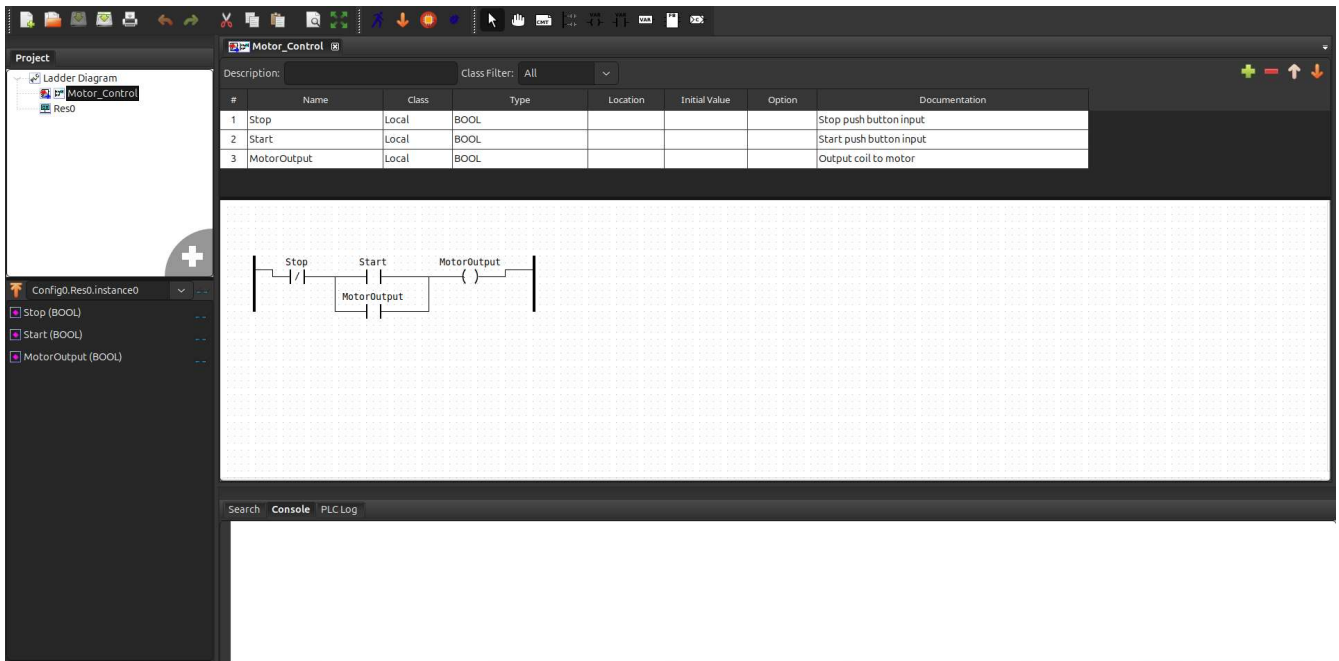
Step 7. Open the Instruction List example and inspect MotorControl. Record the main visual difference between this low-level textual representation and Structured Text.

Step 8. Open the Sequential Function Chart example and inspect MotorControl_SFC. Identify how steps/transitions express process sequence.

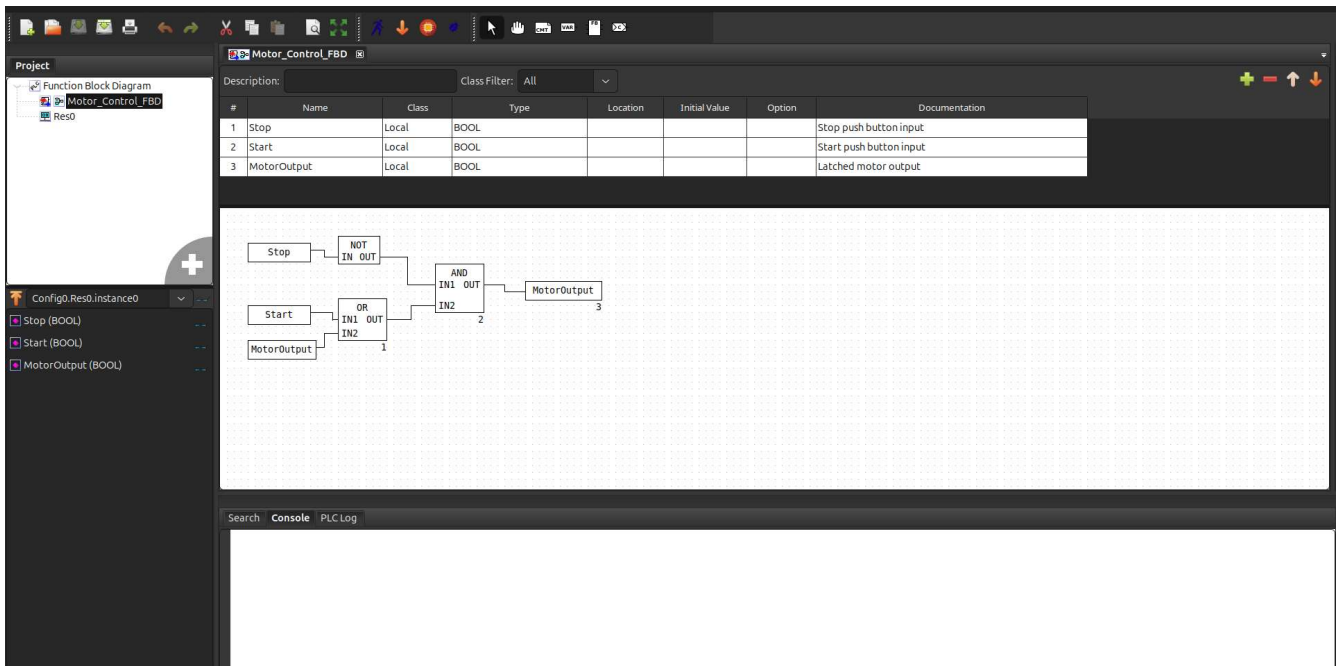
Step 9. Complete the language comparison matrix in the lab report for all five representations.

Step 10. Capture one evidence screenshot for each programming representation, ensuring the program name is visible.

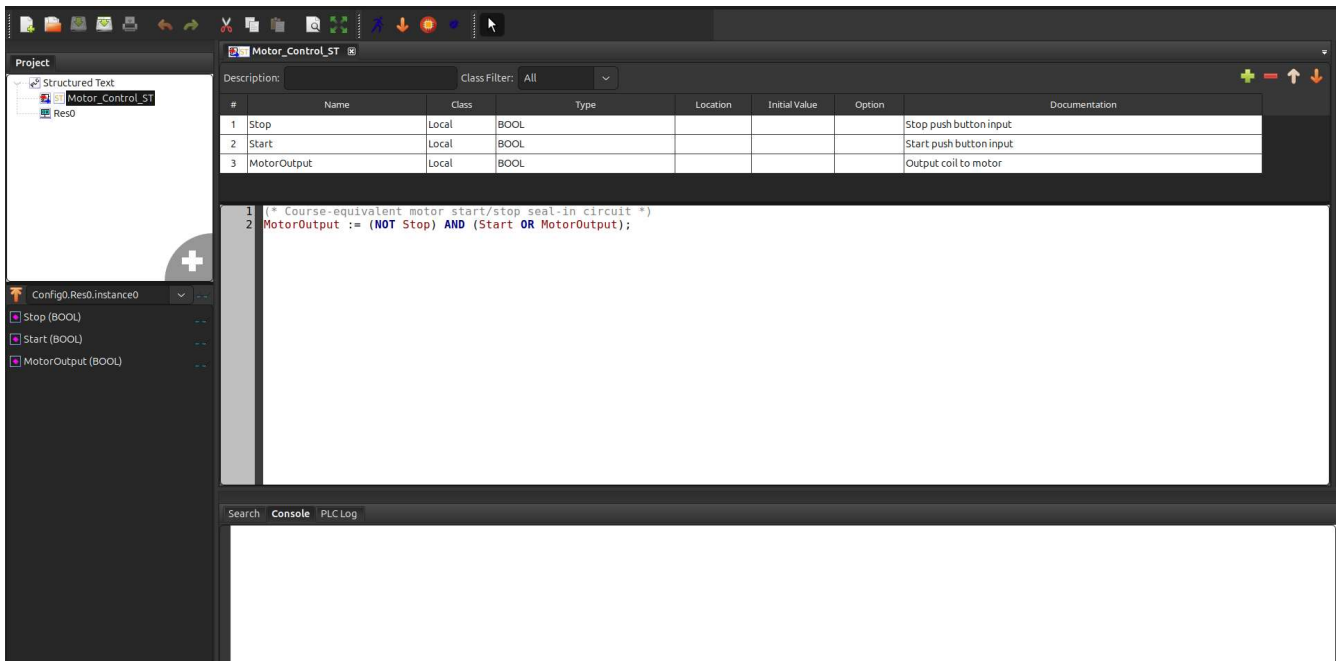
INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)
INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS



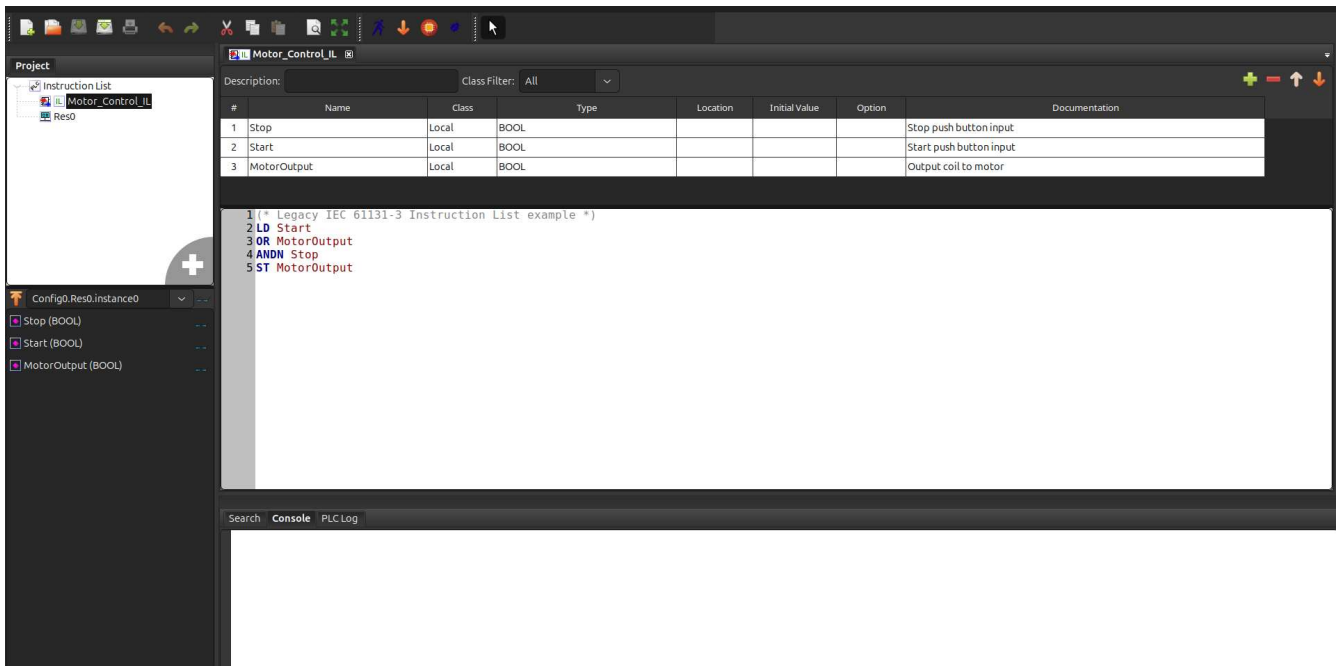
IMG: Ladder Diagram.



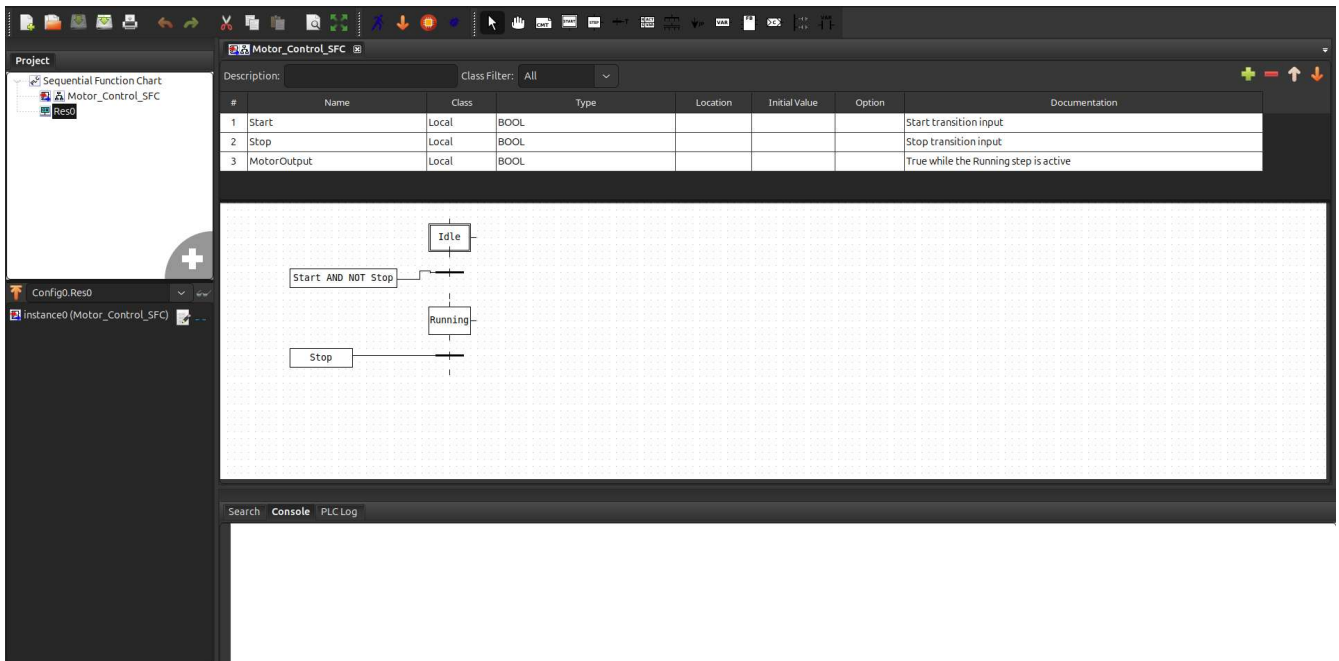
IMG: Function Block diagram.



IMG: Structured Text.



IMG: Instruction List.



IMG: Sequential Function Chart.

IEC Standard 61131 PLC Programming Languages Comparison Matrix

Language	Representation	Strengths	Limitations	Suitable Industrial Use Case
Ladder Diagram (LD)	Graphical (Rungs, Contacts, Coils)	Highly intuitive for electrical maintenance personnel; excellent for real-time online troubleshooting and visualizing binary state flow.	Inefficient for complex mathematical calculations, data array manipulation, or large-scale loop processing.	Discrete manufacturing, motor control centers, and simple conveyor interlocking.
Function Block Diagram (FBD)	Graphical (Blocks linked by signal lines)	Visualizes data and signal flow cleanly; encapsulates complex logic into reusable blocks; great for continuous control.	Complex logic can turn into visually cluttered code that is difficult to trace on small screens.	Process automation, PID temperature/flow control loops, and analog signal scaling.

Structured Text (ST)	Textual (High-level, Python-like syntax)	Highly compact for complex math, data manipulation, arrays, and standard programming constructs (IF/THEN, FOR loops).	Difficult for non-programmers or traditional electricians to read and troubleshoot during live mechanical failures.	Complex data processing, recipe management, string manipulation, and algorithm execution.
Instruction List (IL)	Textual (Low-level, Assembly-like syntax)	Extremely efficient memory usage and fast execution times; highly deterministic.	Very difficult to read, write, and maintain.	Legacy systems maintenance or highly resource-constrained micro-PLCs.
Sequential Function Chart (SFC)	Graphical (Steps, Transitions, Branches)	Excellent for structuring state machines; visually isolates the active process step, making sequence halts easy to diagnose.	Overkill for simple continuous logic; requires underlying LD or ST to program the actual actions within the steps.	Batch processing, automated assembly sequences, and multi-state machine operations (e.g., Fill → Heat → Mix → Drain).

A short explanation showing how the same motor start/stop requirement can be represented differently while preserving the same control objective.

Even though the visual layout changes, the underlying logic for a motor start/stop circuit remains identical across PLC languages because the end goal is always the same: turn the motor on, keep it running, and shut it down when commanded. In a Ladder Diagram, you draw this using parallel and series electrical contacts to build a physical-looking latching circuit. In Structured Text, you write that exact same logic as a simple text equation, such as `MotorOutput := (Start OR MotorOutput) AND NOT Stop;`. A Function Block Diagram takes yet another approach, visually wiring the start and stop signals through standard OR and AND logic blocks on the screen. Ultimately, no matter which visual representation you choose, the PLC compiler translates all of these methods into the exact same low-level machine code to control the physical hardware.

Required Evidence & Submission

- Five screenshots: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- A completed comparison matrix covering representation, strengths, limitations and a suitable industrial use case.
- A short explanation showing how the same motor start/stop requirement can be represented differently while preserving the same control objective.

Recommended filename: <RegistrationNo>_LAB02_PLC_Programming_Languages.pdf

Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: The Ladder Diagram is intuitive for technicians familiar with relay logic because it is a graphical based language that mimics the ladder logic used by electrical technicians when installing relay circuits.

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: Structured Text mimics the style and syntax of high-level programming languages which makes it easier to understand for people familiar with high level programming. It also makes it possible to perform the same function with less code when compared to Ladder Diagram or Function Block Diagram.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: A Sequential function chart might be more appropriate than a single ladder rung when a process is highly sequential and state driven. A ladder rung is evaluated continuously thereby requiring complex interlocking logic to prevent steps from executing out of order. The sequential function chart only executes the logic within the currently active step.

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: The meaning of tags must remain consistent when the representation/language changes because the tag acts as the abstraction layer between the software logic and the physical hardware memory addresses. If the meaning or data type of the tag changes when switching languages, the logic will decouple from the intended hardware, leading to unpredictable, potentially dangerous physical behavior on the machine.

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.
Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.		